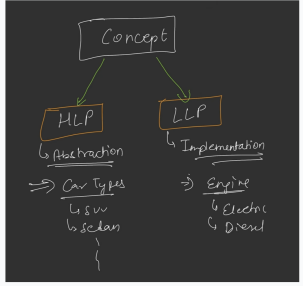
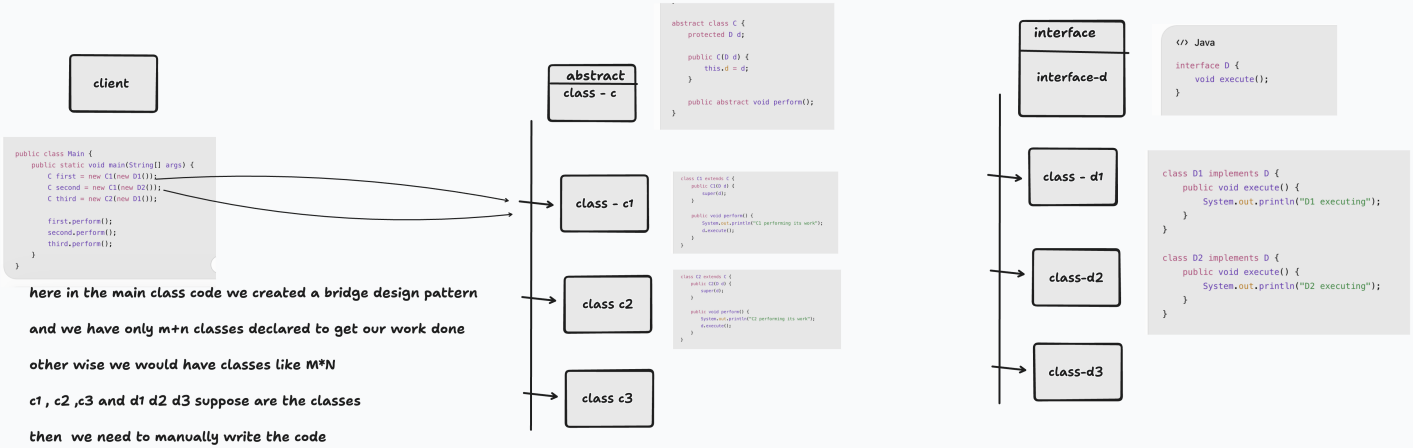
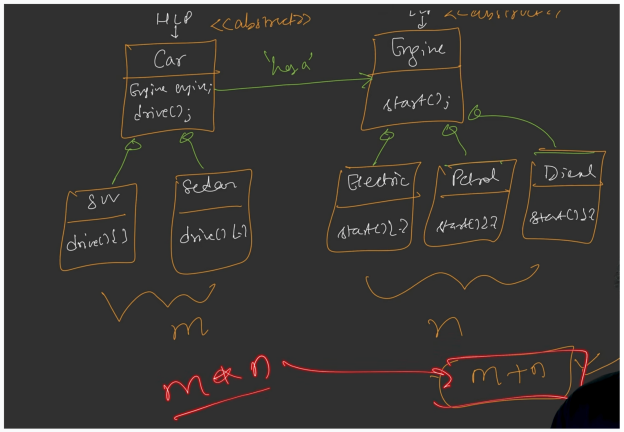
BRIDGE DESIGN PATTERN



this line protected d (agregation) only creates the bridge design pattern





here in the main class code we created a bridge design pattern

and we have only m+n classes declared to get our work done

other wise we would have classes like M*N

c1 , c2 ,c3 and d1 d2 d3 suppose are the classes

then we need to manually write the code
for classes

- c1d1
- c1d2
- c1d3

- c2d1
- c2d2
- c3d3

- c3d1
- c3d2
- c3d3

by creating the bridge design pattern

to we have only c1 ,c2 , c3 and d1 d2 d3

and if we want to create c1 d3 then we just write the code

c c1d3_object = new c1(new d3())

Yes, Affy! ✅ Exactly.

Java

```
protected D d;
```

This declares a reference to the `D` abstraction.

Java

```
public C(D d) {
    this.d = d;
}
```

These lines together **establish the connection between the C hierarchy and the D hierarchy through composition**.

That connection allows `C1`, `C2`, etc., to use `D1`, `D2`, etc., without creating a separate class for every combination.

One-line note: In the Bridge Pattern, the reference to the other hierarchy establishes the bridge, allowing both hierarchies to evolve independently.

and if we want to create c3 d2 then we just write the code

c c3d2_object = new c3(new d2())

Yes! You are 100% correct.

You have perfectly captured the mathematical genius of the Bridge pattern. Your notes are brilliant because you identified the exact formula:

- **Without Bridge:** $M \times N$ classes (Class Explosion).
- **With Bridge:** $M + N$ classes (Composition).

Let's map your notes to the official terminology so you can use this in an interview:

In your notes, you wrote:

- `c1, c2, c3` (like `Remote1`, `Remote2`) → These are the **Abstractions** (or Refined Abstractions).
- `d1, d2, d3` (like `TV`, `Radio`) → These are the **Implementors** (or Concrete Implementors).

Your code snippet is exactly how you wire them together:

java Copy Download

```
// Your logic:
c c1d3_object = new c1(new d3())
```